



FOR JAVA DEVELOPERS WHO HAVE NEVER TOUCHED THE NODE ECOSYSTEM

The Toolchain, From Nothing Installed

Spring Boot to TypeScript, episode 0

Every tutorial starts at minute three, on a machine that already has everything.

This is the part before that. Node, npm, npx, TypeScript, tsconfig, and only then NestJS, in that order, with nothing assumed.

The demonstration that matters is halfway through: compile a TypeScript file, open the JavaScript it produced, and watch the type disappear. Node runs JavaScript and has never seen TypeScript in its life.

Every version on this sheet was current on 19 September 2026 and will not stay current. The method is the durable part: check the source, run the command, and write down what actually came back.

In scope

- The layers: TypeScript compiles to JavaScript, which runs on Node
- Choosing a Node version deliberately, and a version manager
- A plain Node project before any framework
- TypeScript as a project-local dev dependency, and what npx is
- tsconfig, and only the options this backend actually uses
- Compiling, reading the output, and breaking it on purpose
- npm scripts, and where the Maven analogy stops

Out of scope

- Databases and persistence, which come later in the course
- Validation and error handling, which get their own episodes
- Deployment beyond a local docker compose
- Any framework comparison beyond the shapes that transfer



- NestJS last, once the ground under it is understood
- Both applications answering the same endpoint

Installation comes before application code. That ordering is the episode.

01 The versions this course pins, and what most tutorials still say

Node LTS	24.21.0 (tutorials often say 18 or 20)
npm	11.19.0, ships with that Node
TypeScript	7.0.2 (most material assumes 5.x)
Java	25 LTS, pinned in pom.xml
Spring Boot	4.1.1 (Initializr REFUSES 3.5.6)
Everything else	check it on the day you start

Checked against the official sources on the day, not remembered. The right hand column is the trap.

03 The whole install, in the order it was run

Verified on macOS on 18 September 2026. On Linux the sequence is identical once nvm is installed from its own install script. On Windows use nvm-windows, or WSL2 and the Linux path.

the Node path, ending at a verified endpoint

```
git clone https://github.com/code-with-sam-dev/spring-to-node
cd spring-to-node
nvm install && nvm use                # reads .nvmrc → 24.21.0
node --version && npm --version        # v24.21.0, 11.19.0
cd nestjs-api
npm install                          # reads package-lock.json
npm test
npm run build                        # TypeScript → dist/
ls dist                             # the JavaScript Node runs
npm run start:dev
curl -i http://localhost:3000/health # HTTP/1.1 200 {"status":"UP"}
```

the Spring Boot path, same repository

```
cd spring-boot-api
./mvnw test
./mvnw spring-boot:run
curl -i http://localhost:8080/health # HTTP/1.1 200 {"status":"UP"}
```

Java 25 is pinned in pom.xml. An older JAVA_HOME fails with "release version 25 not supported".

or both at once, with Docker

```
docker compose up --build
curl -i http://localhost:8080/health
curl -i http://localhost:3000/health
```

Both applications answered GET /health with HTTP 200 and {"status":"UP"} on the verification run.

Run on a real machine, 2026-09-19. See VERIFIED-INSTALL-LOG.md in the course repository.



04 Four separate things, and people conflate them

You write Java, javac makes bytecode, the JVM runs it, and Spring Boot is a framework on top. The other column has the same shape: you write TypeScript, tsc makes JavaScript, Node runs it, NestJS is a framework on top.

Say it plainly, because getting this wrong is the biggest single source of confusion arriving from Java: Node is not TypeScript. Node is not NestJS. TypeScript is not Node. NestJS is not Node.

You install them separately, in that order.

Node is the runtime. npm is the package manager, and it ships with Node rather than being installed separately.

Run on a real machine, 2026-09-19. See VERIFIED-INSTALL-LOG.md in the course repository.

The JVM-and-Maven comparison is approximate: Maven is a build tool with a lifecycle, npm is closer to a dependency installer with a script runner attached.

A judgement, not a documented fact.



05 Check the version, do not copy it

The official download page gives the current long term support release. On the day this was recorded that was 24.21.0, and installing it brought npm 11.19.0 with it.

Install through a version manager rather than a system installer, because one project will eventually need a different version from another. If SDKMAN is your instinct from Java, that instinct is right even though the tools are not equivalent.

Write both numbers in your README, and commit a version file so the repository helps the next person onto the right one.

```
● ● ● is Node there, and which one
```

```
node --version
npm --version
```

```
● ● ● install it through a version manager
```

```
nvm install 24
Downloading and installing node v24.21.0...
Checksums matched!
Now using node v24.21.0 (npm v11.19.0)

nvm itself comes from its own install script: https://github.com/nvm-sh/nvm
```

Node 24.21.0 was the current LTS, and 26.9.0 the current release, on 19 September 2026.

nodejs.org/en/download, checked 2026-09-19

nvm install 24 reported: Now using node v24.21.0 (npm v11.19.0).

Run on a real machine, 2026-09-19. See VERIFIED-INSTALL-LOG.md in the course repository.

06 The four second demonstration

Write a file with a Payment type: an id, an amount that is a number, a currency. Compile it. Open the JavaScript that came out.

The type is gone. Not renamed, not compiled into a class, not stored elsewhere. There is no Payment type in that file, because there is no such thing at runtime.

Then break it on purpose. Put a string where the number belongs and the compiler stops you with TS2322. Fix it, compile again, and the generated JavaScript is identical to before. The types worked, and then they vanished.

● ● ● TypeScript into the project, never onto the machine

```
npm init -y
npm install --save-dev typescript
npx tsc --version                # Version 7.0.2
npx tsc --init
npx tsc
ls dist                          # index.js  index.js.map
```

● ● ● dist/index.js: the type is gone

```
"use strict";
Object.defineProperty(exports, "__esModule", { value: true });
const payment = {
    id: 'pay_001',
    amount: 100,
    currency: 'USD',
};
console.log(`Payment ${payment.id} for ${payment.amount} ${payment.currency}`);
```

● ● ● break it on purpose: a string where a number belongs

```
npx tsc
src/index.ts(9,3): error TS2322: Type 'string' is not assignable to type 'number'.
```

After compiling, dist/index.js contains the object literal and the log line, and no trace of the declared type.

Run on a real machine, 2026-09-19. See VERIFIED-INSTALL-LOG.md in the course repository.

The deliberate error reproduces as: error TS2322: Type 'string' is not assignable to type 'number'.

Run on a real machine, 2026-09-19. See VERIFIED-INSTALL-LOG.md in the course repository.

Because annotations are erased, data arriving at runtime is unchecked. Validation at the edges is separate code you have to write.

A judgement, not a documented fact.

07 Four things that will make you think you broke something

Run `npm init -y` in a directory called `node-basics` and the package is named `"basics"`. `npm` strips the `node` prefix.

Run `tsc --init` and the generated config sets no root directory and no output directory. The compiler has no idea where to put anything until you say.

Generate a fresh NestJS project and it writes a `vitest` config and an `oxlint` config, not `jest` and `eslint`. Tutorials describing `jest.config` are not wrong, they are old.

Ask Spring Initializr for Boot 3.5.6 and it refuses outright: the compatibility range is 4.0.0 and above.

```
generate the NestJS project, and read what it wrote
```

```
npx @nestjs/cli new nestjs-api
CREATE nestjs-api/.oxlintrc.json (206 bytes)
CREATE nestjs-api/vitest.config.ts (350 bytes)
CREATE nestjs-api/vitest.config.e2e.ts (245 bytes)
CREATE nestjs-api/src/main.ts (237 bytes)
CREATE nestjs-api/src/app.module.ts (255 bytes)
```

`npm init -y` in `node-basics` produced `"name": "basics"`.

Run on a real machine, 2026-09-19. See `VERIFIED-INSTALL-LOG.md` in the course repository.

A fresh Nest project generated `vitest.config.ts`, `vitest.config.e2e.ts` and `.oxlintrc.json`.

Run on a real machine, 2026-09-19. See `VERIFIED-INSTALL-LOG.md` in the course repository.

Initializr returned: Invalid Spring Boot version '3.5.6', Spring Boot compatibility range is `>=4.0.0`.

`start.spring.io` metadata endpoint, checked 2026-09-19

Initializr version ids carry a legacy `.RELEASE` suffix that is not the Maven coordinate. Passing `4.1.1.RELEASE` writes an unresolvable parent POM; the coordinate is `4.1.1`.

Run on a real machine, 2026-09-19. See `VERIFIED-INSTALL-LOG.md` in the course repository.



08 Ten thousand six hundred and seventy five, against forty six

`node_modules` is generated, enormous, and rebuildable from one command, so it never goes into version control.

Measured on the repository for this course rather than asserted: without an ignore file git offered to track 10,675 files. With one, 46.

The lock file is the opposite case. It records the exact versions that were installed and it is what makes a clone reproducible, so that one is committed.

git offered to track 10,675 files without `.gitignore` and 46 with it, in the same project, in the same minute.

Run on a real machine, 2026-09-19. See `VERIFIED-INSTALL-LOG.md` in the course repository.

A fresh clone then runs: `mvn test` 2 passing, `npm ci`, `npm test` 2 passing.

Run on a real machine, 2026-09-19. See `VERIFIED-INSTALL-LOG.md` in the course repository.

Before you write any application code

1. `node --version` and `npm --version` both answer
2. You opened the REPL and ran something in it
3. Your Node version came from the official site today, not a blog post
4. A version file is committed so the repository helps the next person
5. TypeScript is a dev dependency, not a global install
6. You have opened the generated JavaScript at least once
7. You have seen a compile error and fixed it
8. `node_modules` is ignored and the lock file is committed
9. A fresh clone builds and tests without you explaining anything

Everything else



Site

<https://code-with-sam-dev.github.io>



YouTube

<https://www.youtube.com/@CodewithSam-Dev>



LinkedIn

<https://www.linkedin.com/company/code-with-sam-dev>



x

<https://x.com/CodeWithSamDev>



TikTok

<https://www.tiktok.com/@codewithsamdev>



Instagram

<https://www.instagram.com/codewithsamdev>



Facebook

<https://www.facebook.com/codewithsamdev>



GitHub

<https://github.com/code-with-sam-dev>



Buy me a coffee

<https://buymeacoffee.com/codewithsam>



Buy me sadza

<https://buymesadza.com/codewithsam>

Node.js downloads

<https://nodejs.org/en/download>

NestJS docs

<https://docs.nestjs.com>

Spring Initializr

<https://start.spring.io>

If this was useful, a like and a subscribe help more than you would think. And tell me in the comments which part of your workflow you want taken apart next.

Node.js is a trademark of the OpenJS Foundation. Java is a trademark of Oracle. Spring and Spring Boot are trademarks of Broadcom. NestJS and TypeScript are the property of their respective owners. This is an independent, unofficial guide produced by Code with Sam, not affiliated with or endorsed by any of them, and no third party artwork is reproduced here.

Technical claims verified 2026-09-19 against the documentation named beside each one. Vendors change behaviour and defaults between versions, so check the version you are actually running before you rely on any of it.